Lecture 11

Game Programming Basics

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem

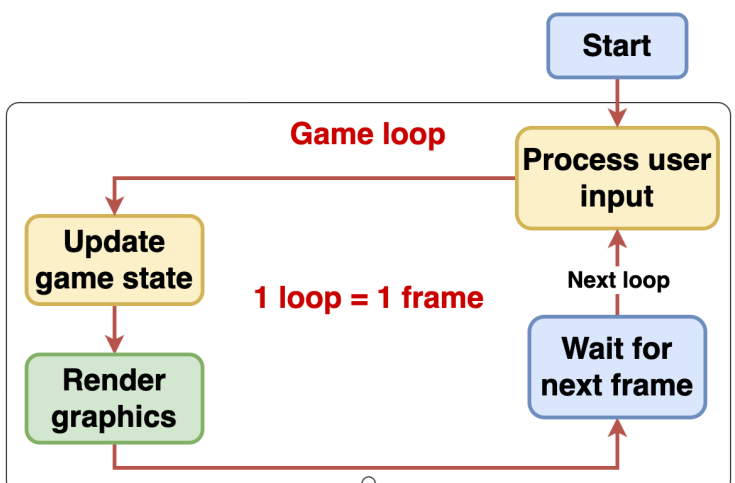
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
  - (we’re working on getting the other TV fixed…)
- **Recording?**
- Important announcements?

Game programming

- Usually graphical instead of text-based
- I/O is usually *nonblocking*
- Usually has a *game loop*



Game programming

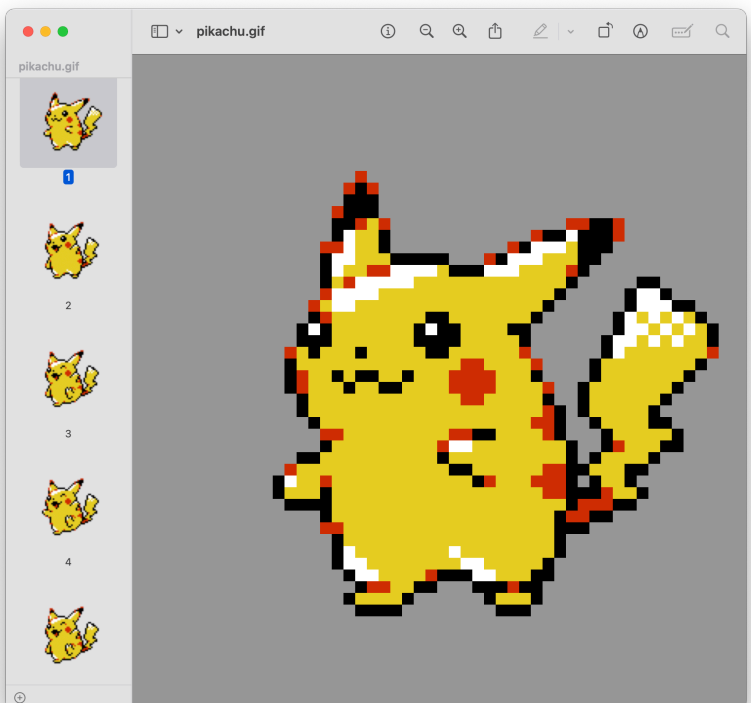
- More complex games may have separate loops for simulation and rendering

Frame rate

Frame rate: Number of frames per unit time (usually FPS/“frames per second”)

- Animation = sequence of still images/frames

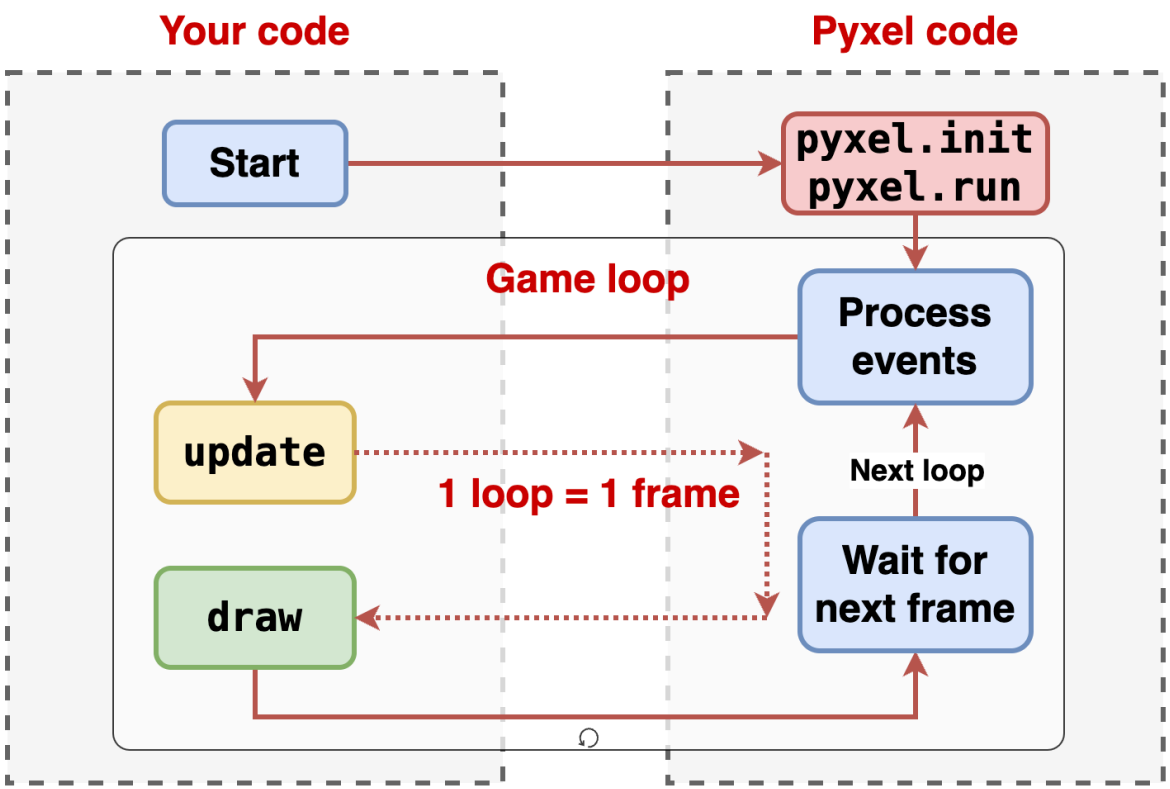
Frame rate



Pyxel

- *Pyxel*: Python retro (pixel-based) game engine
- `https://github.com/kitao/pyxel`
  - compatible with Pyright strict mode
- Platforms:
  - Desktop (Windows, macOS, Linux)
  - Web (supports mobile browsers)
- Installation:
  - `pip install pyxel`

Game loop in Pyxel



Game loop in Pyxel

Inversion of control: Flow of control is *received* by your code

- Usually: Flow of control is given **by** your code to a library
- Inverted: Flow of control is given **to** your code by Pyxel framework

Pyxel app startup

- Empty app (`main.py`):

```
import pyxel

def update():
    pass

def draw():
    pass

pyxel.init(100, 100)
pyxel.run(update, draw)
```

Pyxel app startup

- `pyxel.init(100, 100)` initializes an empty 100×100 screen (not 1:1 with screen pixels)
  - `fps` parameter: Frames (still images) per second (default 30)
- `update` and `draw` functions should be defined
  - `update`: Defines how game state changes
  - `draw`: Defines how screen changes
  - Called once per frame (update $\Rightarrow$ draw)
- `pyxel.run` should be called with `update` and `draw`

Running Pyxel apps

- Run as desktop app:
  - Run as regular Python program
  - `pyxel run <file>`
- Run as web app:
  - `pyxel package . <filename>.py`
  - `pyxel app2html <filename>.pyxapp`

Hello world in Pyxel

```
import pyxel

def update():
    pass

def draw():
    pyxel.text(0, 0, 'Hello, world!', 1)

pyxel.init(100, 100)
pyxel.run(update, draw)
```

Hello world in Pyxel

```
pyxel.text(x: int, y: int, text: str, color: int)
```

- Renders text on screen with top-left corner at (x, y)
- 16 colors (`0` to `15`; link)

Drawing with Pyxel

- `pyxel.circ(x: float, y: float, r: float, color: int)`
  - Renders a filled circle with center at (x, y) (`circb` for unfilled)
- `pyxel.rect(x: float, y: float, w: float, h: float, color: int)`
  - Renders a filled rectangle with top-left corner at (x, y) (`rectb` for unfilled)
- `pyxel.cls(color: int)`
  - Fills screen with given color
- Full list: link

Taking input with Pyxel

- `pyxel.btn(key: int) -> bool`
  - Returns whether key is currently pressed
- `pyxel.btnp(key: int) -> bool`
  - Returns whether key was pressed during current frame
- `pyxel.btnr(key: int) -> bool`
  - Returns whether key was released during current frame
- Full list: link
  - Mouse interaction functions also available

Pyxel + sample Flappy Bird demo